On Demand CMake Course

Bill Hoffman

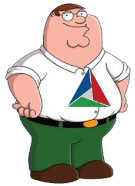
kitware

About Me

- **1990-1999 GE research in a Computer Vision group.**
  - Moved them from Symbolics Lisp Machines to C++ (CFront) on Solaris and later Linux/Windows
  - Was the build and software library guy (gmake/autotools)



- **1998-Present Founder and CTO Kitware**
  - Started CMake in 1999
  - Mostly management now (pointy haired boss stuff), keeping CMake alive and well





kitware



The CMake Company

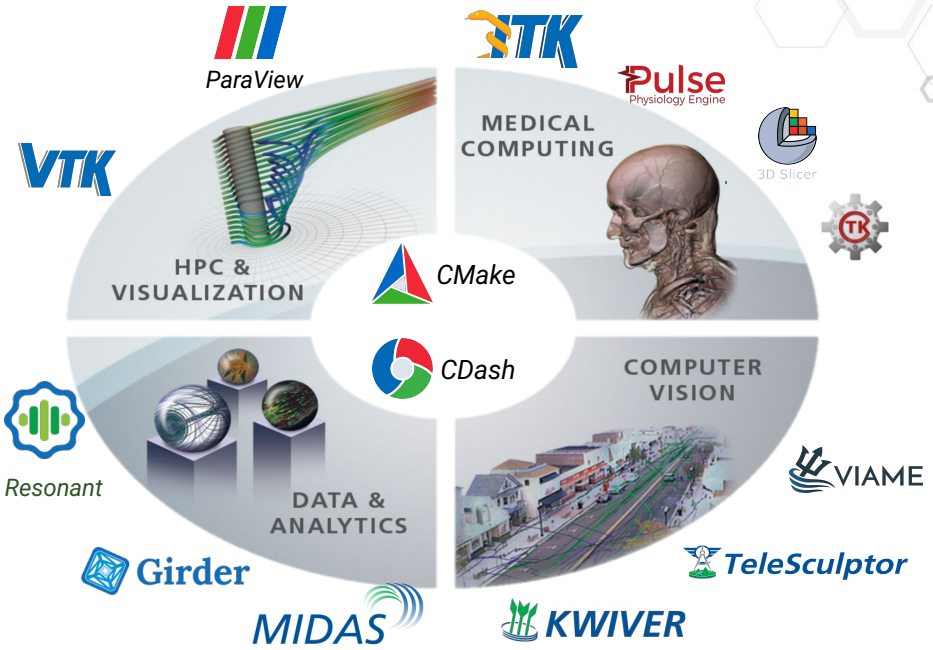
All CMake all the time!

100% Employee Owned

kitware

Collaborative software R&D

- Technical computing
- Algorithms & applications
- Software process & infrastructure
- Support & training
- Open source leadership

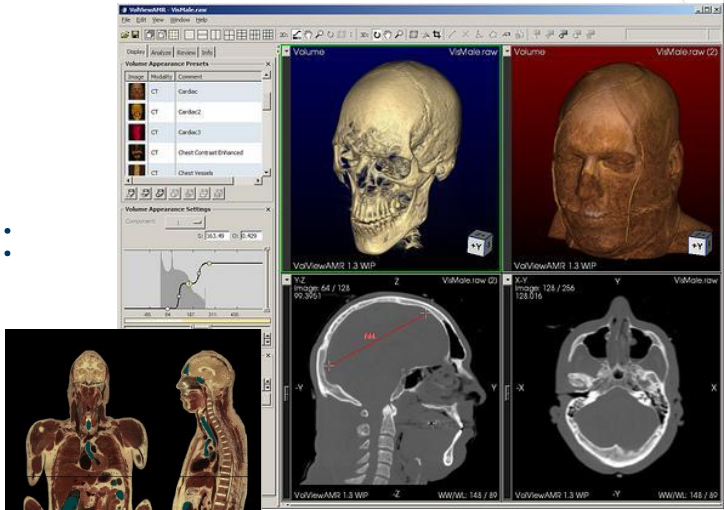


kitware

100% Employee Owned

Where did CMake come from?

- Kitware was the lead engineering team for the Insight Segmentation and Registration Toolkit (ITK) http://www.itk.org
  - tasked with making it build on Unix/Windows/Mac
- Funded by National Library of Medicine (NLM): part of the Visible Human Project
  - Data CT/MR/Slice 1994/1995
  - Code (ITK) 1999
- CMake Release-1-0 branch created in 2001
- Since then funding from many projects and outside contributions





kitware

Happy Birthday CMake!

August 31, 2015 Bill Hoffman



The CMake project officially launched on this day in 2000 on the Insight Segmentation and Registration Toolkit (ITK) mailing list:

http://public.kitware.com/pipermail/insight-developers/2000-August/024248.html

A small, five-person company, called Kitware (here is our 2000 web page), was taking part in a project funded by the National Library of Medicine to create a cross-platform C++ toolkit that would codify the state-of-the-art in segmentation and registration algorithms and complement the data in the Visible Human project. One of Kitware's tasks was to create a build system that would work on Unix, Windows, and Mac OS. Kitware's founding project, VTK, had a build system called pcmaker, which used autotools on Unix and a C++ program that created nmake makefiles on Windows. CMake was created with the limitations of that tool in mind.

One of the main objectives of CMake was to not add external dependencies to the project. Since CMake was used to build C++, the only thing on which we could depend would be a valid C++ compiler. That is why CMake is written entirely in C/C++, so that it can be built on any system on which you need to build your C++ project. This avoids what I like to call the Easter egg hunt build process. First find/build/install these five things and then you are ready to build the project you actually want to use.

kitware

about solutions expertise news careers contact EU Enter Keyword

news › blog › post

25 Years! Yikes, I was 33!

Happy Birthday CMake!

August 31, 2015 Bill Hoffman



The CMake project officially launched on this day in 2000 on the Insight Segmentation and Registration Toolkit (ITK) mailing list:

http://public.kitware.com/pipermail/insight-developers/2000-August/024248.html

A small, five-person company, called Kitware (here is our 2000 web page), was taking part in a project funded by the National Library of Medicine to create a cross-platform C++ toolkit that would codify the state-of-the-art in segmentation and registration algorithms and complement the data in the Visible Human project. One of Kitware's tasks was to create a build system that would work on Unix, Windows, and Mac OS. Kitware's founding project, VTK, had a build system called pcmaker, which used autotools on Unix and a C++ program that created nmake makefiles on Windows. CMake was created with the limitations of that tool in mind.

One of the main objectives of CMake was to not add external dependencies to the project. Since CMake was used to build C++, the only thing on which we could depend would be a valid C++ compiler. That is why CMake is written entirely in C/C++, so that it can be built on any system on which you need to build your C++ project. This avoids what I like to call the Easter egg hunt build process. First find/build/install these five things and then you are ready to build the project you actually want to use.

kitware

about solutions expertise news careers contact EU Enter Keyword

news › blog › post

25 Years! Yikes, I was 33!

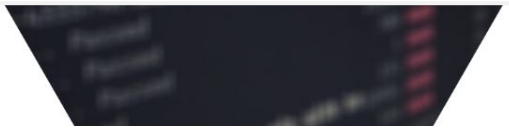
Happy Birthday CMake!

August 31, 2015 Bill Hoffman



One of the main objectives of CMake was to not add external dependencies to the project. Since CMake was used to build C++, the only thing on which we could depend would be a valid C++ compiler. That is why CMake is written entirely in C/C++, so that it can be built on any system on which you need to build your C++ project. This avoids what I like to call the Easter egg hunt build process. First find/build/install these five things and then you are ready to build the project you actually want to use.

registration algorithms and complement the data in the Visible Human project. One of Kitware's tasks was to create a build system that would work on Unix, Windows, and Mac OS. Kitware's founding project, VTK, had a build system called pcmaker, which used autotools on Unix and a C++ program that created nmake makefiles on Windows. CMake was created with the limitations of that tool in mind.



One of the main objectives of CMake was to not add external dependencies to the project. Since CMake was used to build C++, the only thing on which we could depend would be a valid C++ compiler. That is why CMake is written entirely in C/C++, so that it can be built on any system on which you need to build your C++ project. This avoids what I like to call the Easter egg hunt build process. First find/build/install these five things and then you are ready to build the project you actually want to

Happy Birthday CMake!

August 31, 2015 Bill Hoffman



The CMake project officially launched on this day in 2000 on the Insight Segmentation and Registration Toolkit (ITK) mailing list:

http://public.kitware.com/pipermail/insight-developers/2000-August/024248.html

A small, five-person company, called Kitware (here is our 2000 web page), was taking part in a project funded by the National Library of Medicine to create a cross-platform C++ toolkit that would codify the state-of-the-art in segmentation and registration algorithms and complement the data in the Visible Human project. One of Kitware's tasks was to create a build system that would work on Unix, Windows, and Mac OS. Kitware's founding project, VTK, had a build system called pcmaker, which used autotools on Unix and a C++ program that created nmake makefiles on Windows. CMake was created with the limitations of that tool in mind.

One of the main objectives of CMake was to not add external dependencies to the project. Since CMake was used to build C++, the only thing on which we could depend would be a valid C++ compiler. That is why CMake is written entirely in C/C++, so that it can be built on any system on which you need to build your C++ project. This avoids what I like to call the Easter egg hunt build process. First find/build/install these five things and then you are ready to build the project you actually want to use.

kitware

news › blog › post

Happy Birthday CI

August 31, 2015 Bill Hoffm

The CMake project officially launched
list:

http://public.kitware.com/pipermail/in

A small, five-person company, called K
Library of Medicine to create a cross-p
registration algorithms and compleme
system that would work on Unix, Winc
which used autotools on Unix and a C
limitations of that tool in mind.

One of the main objectives of CMak
+, the only thing on which we could
that it can be built on any system on
hunt build process. First find/build/in

public.kitware.com/pipermail/insight-developers/2000-August/024248.html

Kitware Bookmarks | Kitware - Calendar... | Login | 1Password... | Inbox (22) - bill.ho... | K

EU Enter Keyword

[Insight-developers] CMake / ITK build process

Bill Hoffman bill.hoffman at kitware.com
Thu Aug 31 18:29:

[Insight-developers] CMake / ITK build process

- Previous message: [Insight-developers] itkBasicArchitectureTest fails on execut ion
- **Messages sorted by:** [date] [thread] [subject] [author]

I have reworked the build process for ITK.
I have created a package called CMake (Cross platform make).
It is in Insight/CMake, and it includes all the makefile.in
and c++ source for the build process of ITK.

I have also updated and integrated the install documentation with the
InsightStart.doc in Insight/Documents. The InsightStart.doc
is also now available on the main ITK web page from the
top level (as a link):

http://public.kitware.com/Insight/HTMLPrivate/

Here is the direct link to InsightStart on the web:
http://public.kitware.com/Insight/HTMLPrivate/InsightStart.html

(BTW, You can use your cvs login and password to get into the private
web page area.)

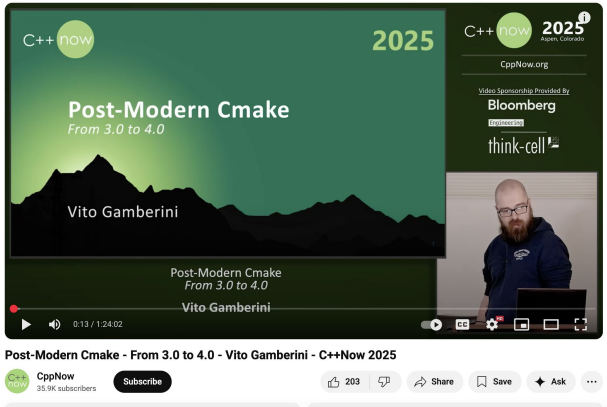
The build process is almost the same as it was. However
a few names have changed (pcbuilder -> CMakeSetup).
Also, Makefile.in is no longer used to list the files you want
built. The file used is CMakeLists.txt, which can no be found
in place of the old Makefile.in files.

As always, let me know if there are any problems.
I still have not fixed the Release build on windows.

-Bill

Eras of CMake

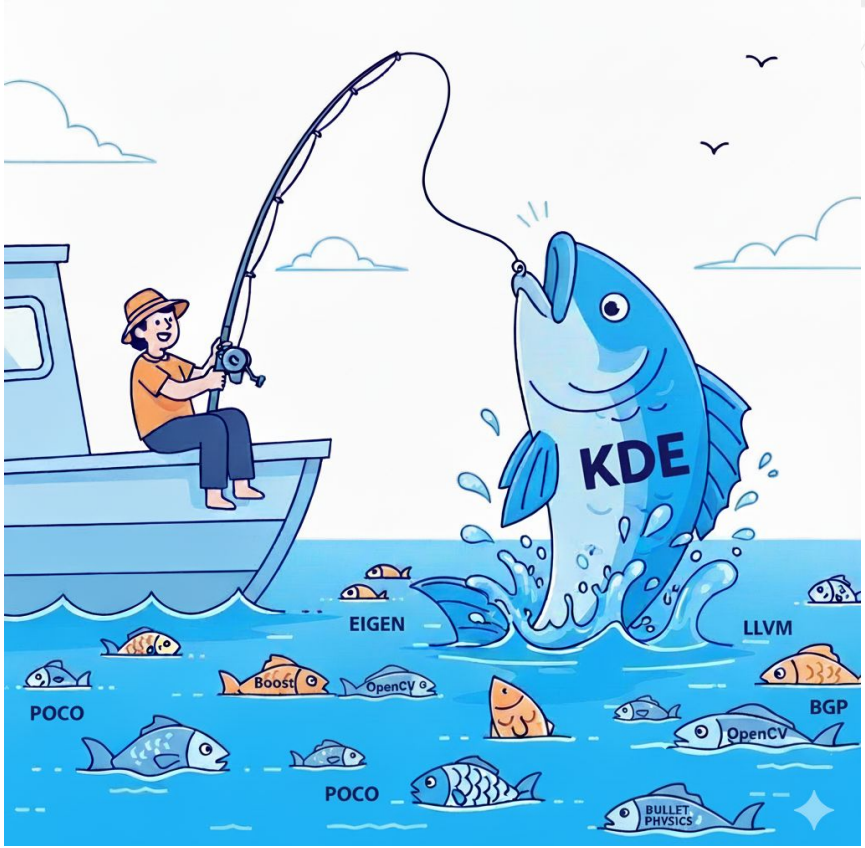
- **CMake 1:** Primordial Flag Soup (2000)
- **CMake 2:** World Domination (2004)
- **CMake 3:** I hope you like target_* commands (2014)
- **CMake 4:** CMake 4: interop and standardization I hope you like json



kitware

CMake 2: World Domination (2004-2014)

- Me fishing for projects to use CMake



kitware

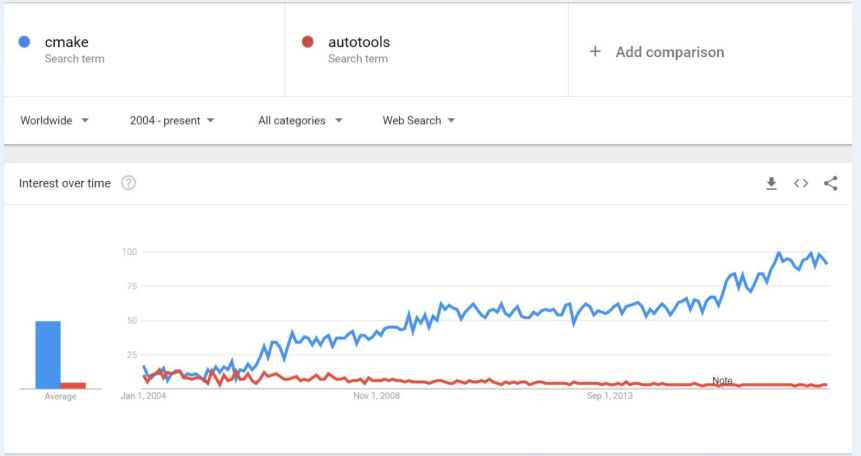
CMake 2: World Domination (2004-2014)

- **2006 KDE**
  - install improvements
    (shared library version links, etc)
  - (cmake) pre-install on linux
  - 2012 Modern CMake
- **2008 LLVM CMake**

2006

KDE switches to CMake

- KDE developer Alexander Neundorf was the champion that helped KDE adopt CMake.
- Kitware worked hard to create a prototype build system in a few short weeks.
- CMake was able to quickly build more of KDE than the scons system they were using previously.
- KDE was able to port applications to Windows and Apple very quickly with CMake.
- CMake got new features, including shared library versioning and installation rpath re-writes without relinking.
- CMake was distributed by all major linux distributions to support KDE.



kitware

CMake 2: World Domination (2004-2014)

2012 Modern CMake
by Stephen Kelly

found here. Then already in 2013, the simple example with Qt shows the essence of Modern CMake:

```
find_package(Qt5Widgets 5.2 REQUIRED)

add_executable(myapp main.cpp)
target_link_libraries(myapp Qt5::Widgets)
```

steveire.wordpress.com/2017/11/05/embracing-modern-cmake/

Kitware - Calendar... | Login | 1Password... | Inbox (22) - bill.ho... | KWiK3 (beta) | Dashboard | Strava | BillHoffmanTrainin...

Design a site like this with WordPress.com Get started

Dreamer, but not the only one.

« Boost dependencies and bcp API Changes in Clang »

Embracing Modern CMake

I spoke at the ACCU conference in April 2017 on the topic of Embracing Modern CMake. The talk was very well attended and received, but was unfortunately not recorded at the event. In September I gave the talk again at the Dublin C++ User Group, so that it could be recorded for the internet.



The slides are available here. The intention of the talk was to present a 'gathered opinion' about what **Modern CMake** is and how it should be written. I got a lot of input from CMake users on reddit which informed some of the content of the talk.

kitware

Let's Learn CMake



kitware